n8n + Ollama + Llama 3.2 3B

Instalación local con Docker Compose

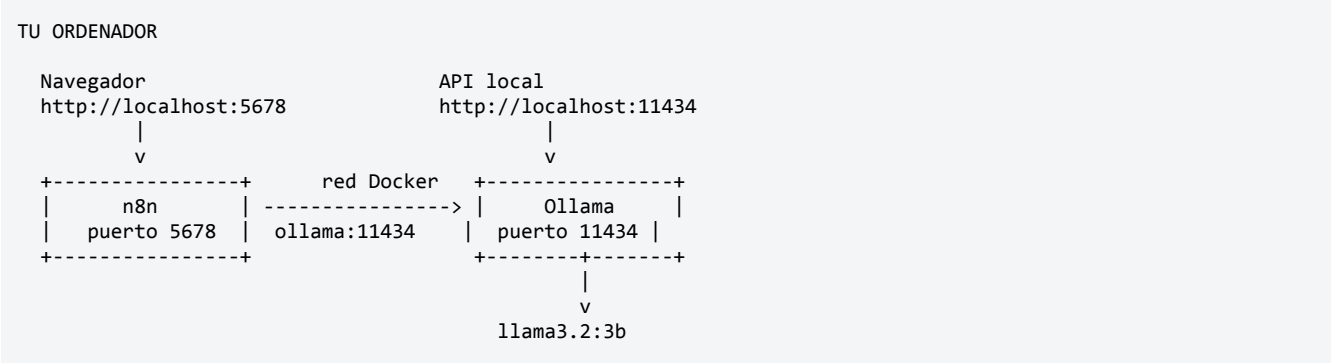
Guía práctica para el alumnado

Objetivo: desplegar n8n y Ollama en contenedores, descargar el modelo local llama3.2:3b, comprobar que ambos servicios funcionan y conectar n8n con Ollama sin utilizar una API de pago.

Modelo utilizado: llama3.2:3b · inferencia local · sin clave API

1. Qué vamos a montar

La práctica crea dos servicios principales dentro de Docker: n8n para automatización de workflows y Ollama para ejecutar un modelo de lenguaje en local. Ambos servicios se comunican mediante una red privada de Docker Compose.



NOTA: Dentro de Docker, n8n no utiliza localhost para llegar a Ollama. Utiliza el nombre del servicio: http://ollama:11434.

2. Requisitos previos

- Docker Desktop instalado y en ejecución (Windows/macOS) o Docker Engine + Docker Compose en Linux.
- Al menos 8 GB de RAM. Con 16 GB el trabajo suele ser más cómodo.
- Unos 5 GB de espacio libre como mínimo para imágenes, volúmenes y el modelo.
- Conexión a Internet solo para descargar las imágenes Docker y el modelo. Después, las consultas al modelo pueden hacerse en local.
- Terminal: PowerShell, Windows Terminal, Bash o similar.

2.1. Comprobar Docker

```
docker --version
docker compose version
```

Si ambos comandos muestran una versión, Docker y Docker Compose están disponibles.

3. Crear el proyecto

Crear una carpeta de trabajo y entrar en ella:

```
mkdir n8n-ollama
cd n8n-ollama
```

Dentro de esa carpeta crear un archivo llamado:

```
docker-compose.yml
```

4. Archivo docker-compose.yml

Copiar el siguiente contenido completo en el archivo. El servicio auxiliar ollama-pull se ejecuta una sola vez para descargar el modelo llama3.2:3b.

```
services:

  n8n:
    image: docker.n8n.io/n8nio/n8n:latest
    container_name: n8n
    restart: unless-stopped

    ports:
      - "127.0.0.1:5678:5678"

    environment:
      - TZ=Europe/Madrid
      - GENERIC_TIMEZONE=Europe/Madrid
      - N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS=true
      - N8N_DIAGNOSTICS_ENABLED=false
      - N8N_PERSONALIZATION_ENABLED=false

    volumes:
      - n8n_data:/home/node/.n8n

    networks:
      - ai-network

    depends_on:
      ollama:
        condition: service_started

  ollama:
    image: ollama/ollama:latest
    container_name: ollama
    restart: unless-stopped

    ports:
      - "127.0.0.1:11434:11434"

    volumes:
      - ollama_data:/root/.ollama

    networks:
      - ai-network

  ollama-pull:
    image: ollama/ollama:latest
    container_name: ollama-pull
    restart: "no"

    environment:
      - OLLAMA_HOST=ollama:11434

    entrypoint: /bin/sh

    command:
      - "-c"
      - |
        echo "Esperando a Ollama..."
        until ollama list > /dev/null 2>&1; do
          sleep 2
        done
        echo "Descargando llama3.2:3b..."
        ollama pull llama3.2:3b
        echo "Modelo instalado."

    networks:
      - ai-network

    depends_on:
      - ollama

volumes:
  n8n_data:
  ollama_data:
```

```
networks:
  ai-network:
    driver: bridge
```

ATENCIÓN: Los volúmenes n8n_data y ollama_data hacen persistentes los workflows, credenciales y modelos. Evita usar docker compose down -v salvo que quieras borrar esos datos.

5. Arrancar los servicios

Desde la carpeta donde se encuentra docker-compose.yml ejecutar:

```
docker compose up -d
```

Docker descargará las imágenes necesarias y el servicio ollama-pull descargará Llama 3.2 3B.

5.1. Ver el estado de los contenedores

```
docker compose ps
```

El resultado esperado es que n8n y ollama estén en estado Up. El contenedor ollama-pull puede aparecer como Exited (0); es normal, porque termina después de descargar el modelo.

5.2. Ver la descarga del modelo

```
docker compose logs -f ollama-pull
```

NOTA: Puedes salir del seguimiento de logs con Ctrl+C. Esto no detiene los contenedores.

6. Comprobaciones de Ollama y del modelo

6.1. Comprobar modelos instalados

```
docker compose exec ollama ollama list
```

Debe aparecer llama3.2:3b. Su descarga habitual ronda los 2 GB.

6.2. Probar el modelo desde la terminal

```
docker compose exec ollama ollama run llama3.2:3b
```

Cuando aparezca el prompt >>>, escribir por ejemplo:

```
Explica qué es Docker en dos frases.
```

Para salir del chat interactivo:

```
/bye
```

6.3. Comprobar la API HTTP de Ollama

Desde el sistema anfitrión, abrir en el navegador:

```
http://localhost:11434/api/tags
```

También puede comprobarse desde PowerShell:

```
Invoke-RestMethod http://localhost:11434/api/tags
```

COMPROBACIÓN: Si la respuesta contiene llama3.2:3b, Ollama está funcionando y la API está accesible.

7. Entrar en n8n

Abrir en el navegador:

```
http://localhost:5678
```

La primera vez n8n solicitará crear la cuenta propietaria de la instancia local. Después se mostrará el editor de workflows.

8. Conectar n8n con Ollama

Los dos contenedores pertenecen a la red ai-network. Docker Compose ofrece resolución DNS interna usando el nombre del servicio.

Desde dónde	URL de Ollama
Desde Windows/Linux anfitrión	http://localhost:11434
Desde el contenedor n8n	http://ollama:11434

ATENCIÓN: En la credencial de Ollama de n8n debes usar http://ollama:11434. No uses http://localhost:11434 dentro de n8n.

8.1. Crear la credencial en n8n

1. Abrir n8n y crear un workflow nuevo.
2. Añadir un nodo AI Agent.
3. En la conexión Chat Model del agente, añadir Ollama Chat Model.
4. Crear una credencial de tipo Ollama.
5. En Base URL introducir http://ollama:11434.

6. Guardar la credencial.

7. Seleccionar el modelo llama3.2:3b.

9. Primer workflow de prueba

Crear el siguiente flujo:

```
Chat Trigger
     |
     v
  AI Agent
     |
     +---- Chat Model ----> Ollama Chat Model
                                llama3.2:3b
```

Probar el chat con una petición breve, por ejemplo:

```
Explica la diferencia entre TCP y UDP en tres puntos.
```

Si el modelo responde, la comunicación n8n → Ollama → Llama funciona correctamente.

10. Prueba alternativa: llamar a Ollama con HTTP Request

Esta prueba permite ver directamente la API utilizada por Ollama, sin el nodo de IA específico.

1. Añadir un nodo Manual Trigger.

2. Añadir un nodo HTTP Request.

3. Configurar Method como POST.

4. Configurar URL como http://ollama:11434/api/chat.

5. Seleccionar Body Content Type = JSON.

6. Introducir el cuerpo JSON siguiente.

```
{
  "model": "llama3.2:3b",
  "messages": [
    {
      "role": "user",
      "content": "Explica qué es DNS en tres frases."
    }
  ],
  "stream": false
}
```

El texto generado se encuentra normalmente en:

```
{{$json.message.content}}
```

11. Probar la API directamente desde PowerShell

```
$body = @{
    model = "llama3.2:3b"
    messages = @(
        @{
            role = "user"
            content = "Explica qué es DHCP en tres frases."
        }
    )
    stream = $false
} | ConvertTo-Json -Depth 5

$response = Invoke-RestMethod `
    -Method Post `
    -Uri "http://localhost:11434/api/chat" `
    -ContentType "application/json" `
    -Body $body

$response.message.content
```

12. Parar, arrancar y mantener el entorno

Acción	Comando
Arrancar	`docker compose up -d`
Parar sin borrar datos	`docker compose down`
Ver estado	`docker compose ps`
Ver todos los logs	`docker compose logs -f`
Logs de n8n	`docker compose logs -f n8n`
Logs de Ollama	`docker compose logs -f ollama`
Listar modelos	`docker compose exec ollama ollama list`
Abrir Llama 3.2	`docker compose exec ollama ollama run llama3.2:3b`
Actualizar imágenes	`docker compose pull && docker compose up -d`

ATENCIÓN: docker compose down -v elimina también los volúmenes. En esta práctica supondría perder los datos persistentes de n8n y los modelos descargados.

13. Problemas frecuentes

Problema	Qué revisar
n8n no abre en el navegador	Comprobar docker compose ps y docker compose logs n8n. Verificar que el puerto 5678 no esté ocupado.
Ollama no responde	Comprobar docker compose ps, después docker compose logs ollama y probar http://localhost:11434/api/tags.
n8n muestra error de conexión con Ollama	Revisar la Base URL. Dentro de n8n debe ser http://ollama:11434, no localhost.
El modelo no aparece	Ejecutar docker compose exec ollama ollama list. Si falta, usar docker compose exec ollama ollama pull llama3.2:3b.
El modelo responde muy lentamente	Reducir la longitud del prompt y de la respuesta. Cerrar aplicaciones pesadas. Si sigue siendo lento, probar llama3.2:1b.

El contenedor ollama-pull está Exited (0)	Es el comportamiento esperado: solo descarga el modelo y finaliza.
El puerto 5678 o 11434 ya está en uso	Detener el programa que usa ese puerto o modificar el puerto izquierdo en docker-compose.yml.

14. Checklist final

☐ Docker y Docker Compose muestran su versión.

☐ Los contenedores n8n y ollama están en estado Up.

☐ ollama list muestra llama3.2:3b.

☐ El modelo responde desde la terminal.

☐ http://localhost:11434/api/tags devuelve información de modelos.

☐ http://localhost:5678 abre n8n.

☐ La credencial de Ollama en n8n usa http://ollama:11434.

☐ El workflow Chat Trigger → AI Agent → Ollama Chat Model responde correctamente.

15. Actividad propuesta

Una vez completada la instalación, realizar la siguiente práctica:

- Crear un workflow con Chat Trigger, AI Agent y Ollama Chat Model.
- Utilizar llama3.2:3b.
- Configurar un mensaje de sistema para que el agente responda como asistente técnico de redes.
- Probar al menos tres preguntas sobre redes, sistemas o programación.
- Crear una segunda prueba usando el nodo HTTP Request contra /api/chat.
- Capturar evidencias de docker compose ps, ollama list y una respuesta correcta de n8n.

Entrega mínima

- Captura del estado de los contenedores.
- Captura donde aparezca llama3.2:3b instalado.
- Captura del workflow de n8n.
- Ejemplo de pregunta y respuesta obtenida con el modelo local.
- Breve explicación de por qué n8n utiliza http://ollama:11434 y no http://localhost:11434.

16. Conceptos que debes saber explicar

Concepto	Definición
Contenedor	Proceso aislado creado a partir de una imagen Docker.
Imagen	Plantilla utilizada para crear contenedores.
Volumen	Almacenamiento persistente separado del ciclo de vida del contenedor.
Red Docker	Red virtual que permite que los contenedores se comuniquen.
Nombre de servicio	Nombre DNS interno que Docker Compose resuelve dentro de la red.
Ollama	Servidor local que descarga y ejecuta modelos de lenguaje y expone una API.
n8n	Plataforma de automatización basada en workflows y nodos.

Modelo local	Modelo que se ejecuta en el propio equipo, sin enviar cada consulta a un proveedor cloud.

17. Referencias oficiales

Documentación de n8n: docs.n8n.io

Biblioteca de Llama 3.2 en Ollama: ollama.com/library/llama3.2

Documentación/API de Ollama: docs.ollama.com

Imagen oficial de Ollama en Docker: hub.docker.com/r/ollama/ollama

NOTA: Para esta práctica se usa llama3.2:3b por ser un modelo pequeño y razonable para equipos de aula. Si el rendimiento sigue siendo insuficiente, puede sustituirse por llama3.2:1b cambiando el nombre del modelo en la descarga y en n8n.